

고급프로젝트 최종 발표

딸기 자동 분류 시스템

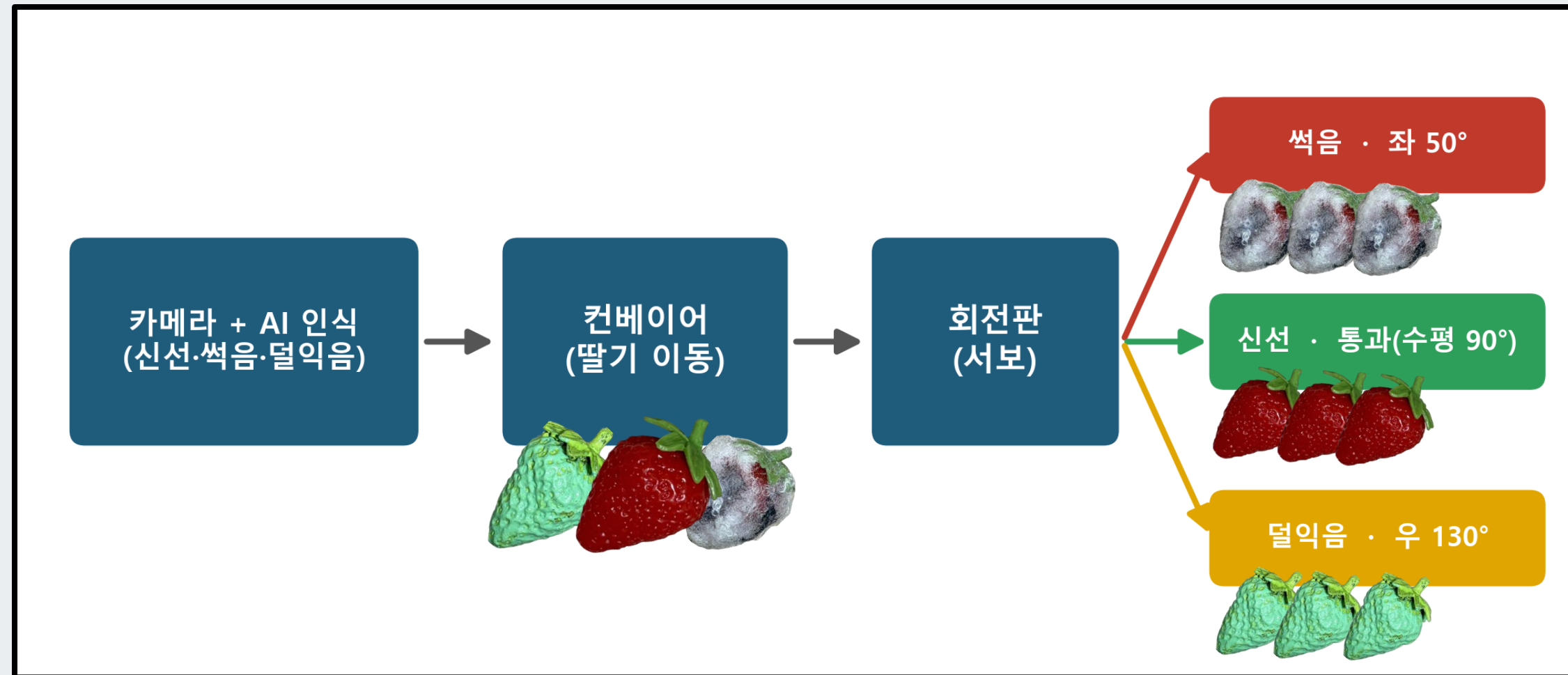
1조
강요한 | 채지훈

목차

01	프로젝트 아이디어
02	구현 방법
03	결과물
04	집중한 부분
05	발전 가능성
06	사용 도구 및 참고자료

1. 프로젝트 아이디어

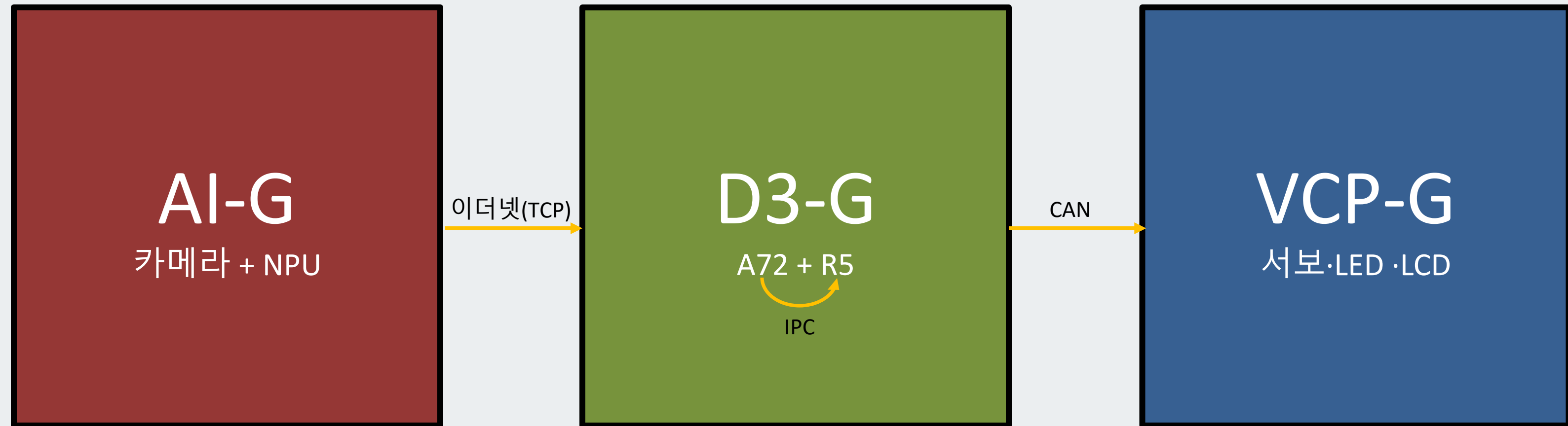
- 딸기 자동 분류 시스템 - "AI가 신선도를 확인하고, 회전판이 자동으로 분류한다"



- 착안: 텔레칩스 자율주행 차량 키트의 기본 뼈대 — 인식(AI-G) → 제어·통신(D3-G) → 구동(VCP-G) — 구조에 착안
- 기능: 사람 없이 AI 모델을 통한 딸기 신선도 판별 + 물리적 분류 → 농산물 선별 자동화

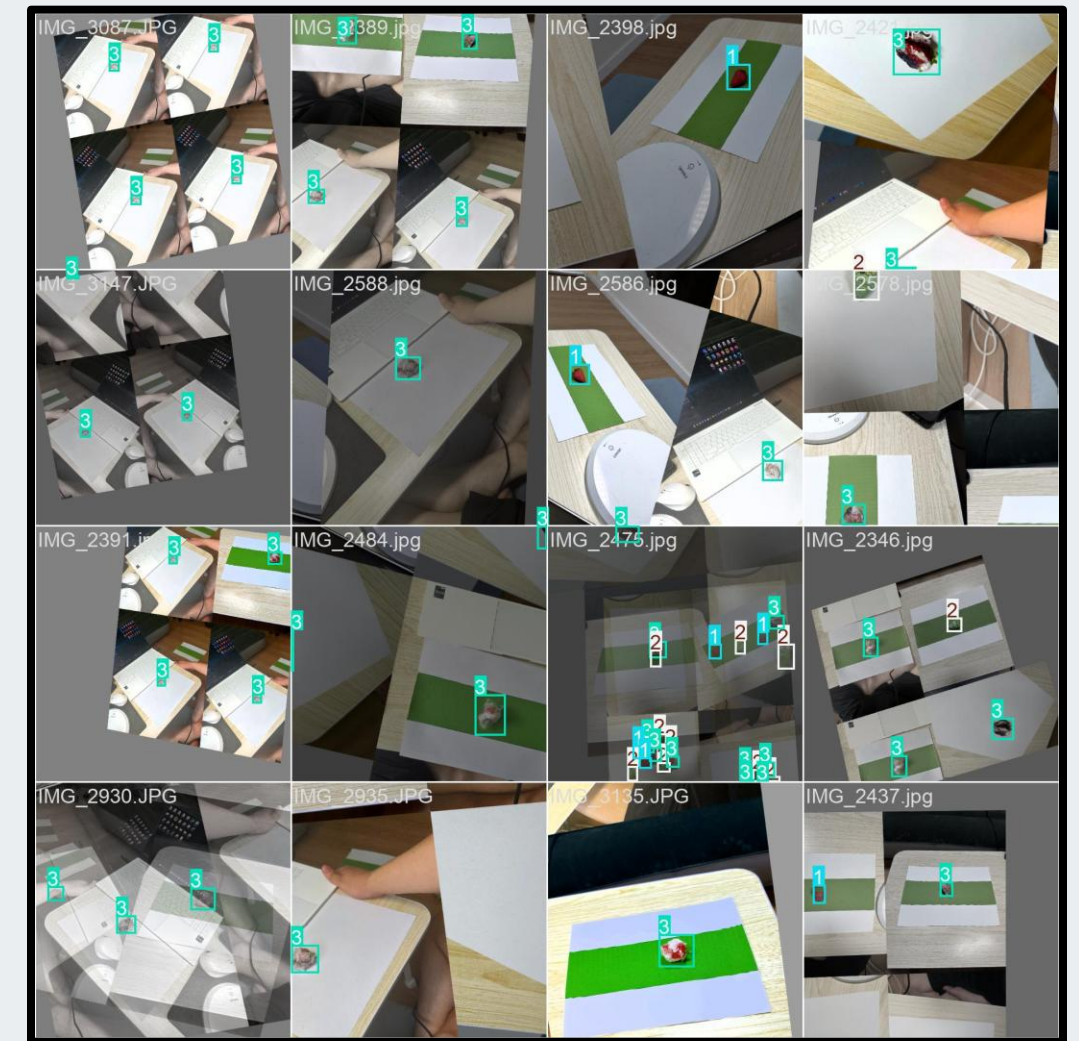
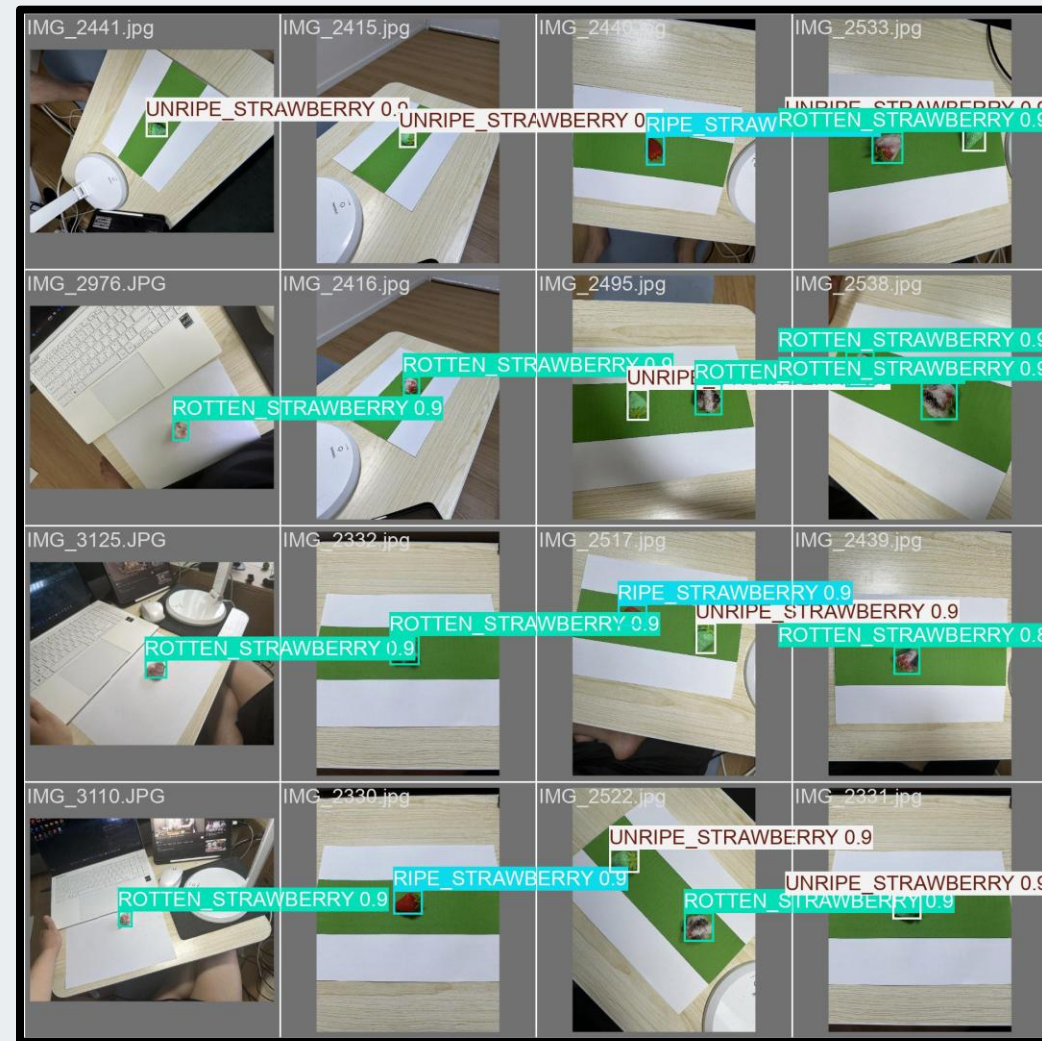
2. 구현 방법

- 시스템 구성: 역할이 다른 3개 보드가 인식 → 제어 → 구동을 나눠 맡아 하나로 연결



- 역할 분담: 인식(AI-G) · 제어·통신(D3-G) · 구동(VCP-G) — 한 보드가 다 하지 않고 나눠 처리 후 통합
- 서로 다른 종류 3보드: NPU(인식) · Linux+RTOS 듀얼코어(제어) · MCU(구동) — 칩·OS·언어가 모두 달라 통합이 핵심 난이도
- 통신 3단: 이더넷(TCP) → IPC(공유메모리) → CAN — 구간마다 다른 방식을 맞물려 연결

2. 구현 방법 – AI 학습·NPU 배포

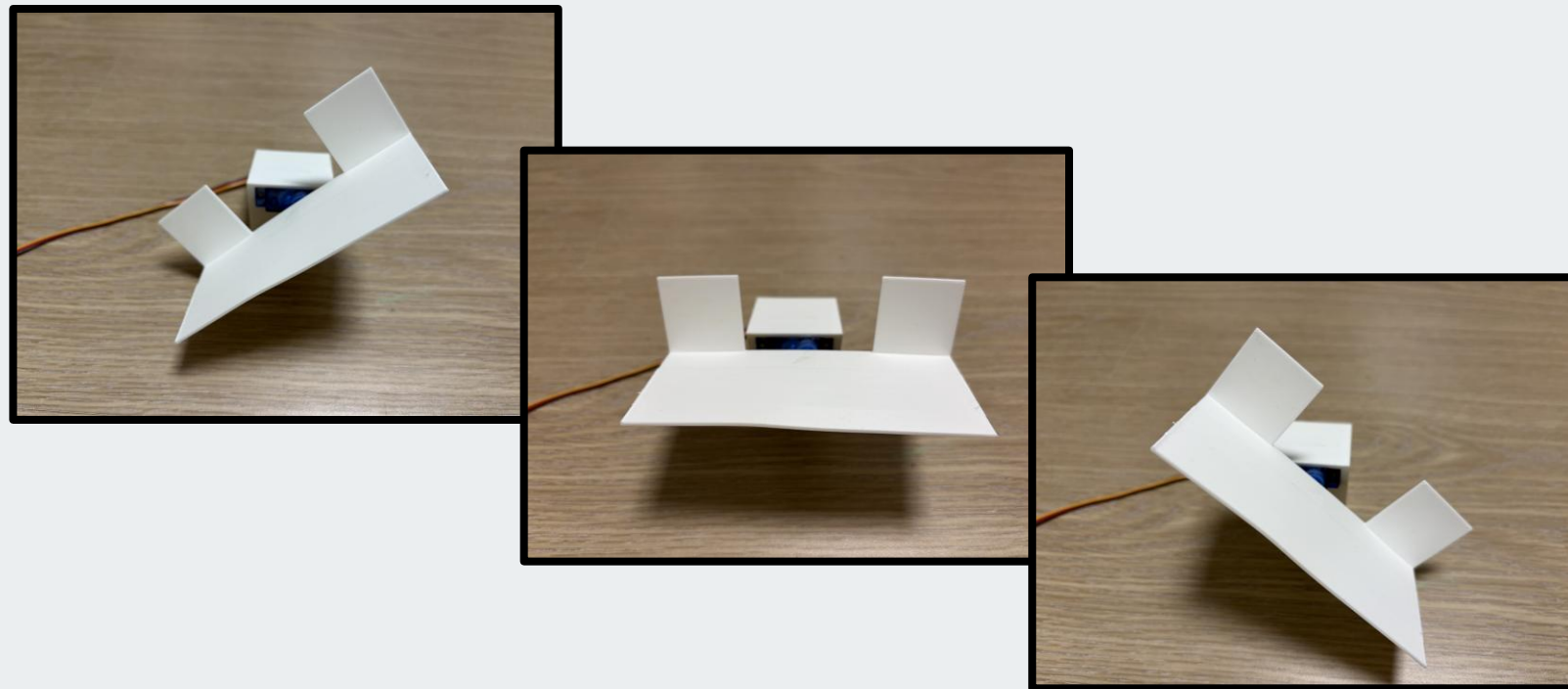


- 데이터셋: 딸기 3종(신선 · 썩음 · 덜익음) 직접 촬영 + 라벨링
- 모델 학습: 경량 모델 YOLOv8n으로 3종 분류 학습
- 양자화·배포: NPU에서 돌도록 INT8 양자화 후 보드 탑재

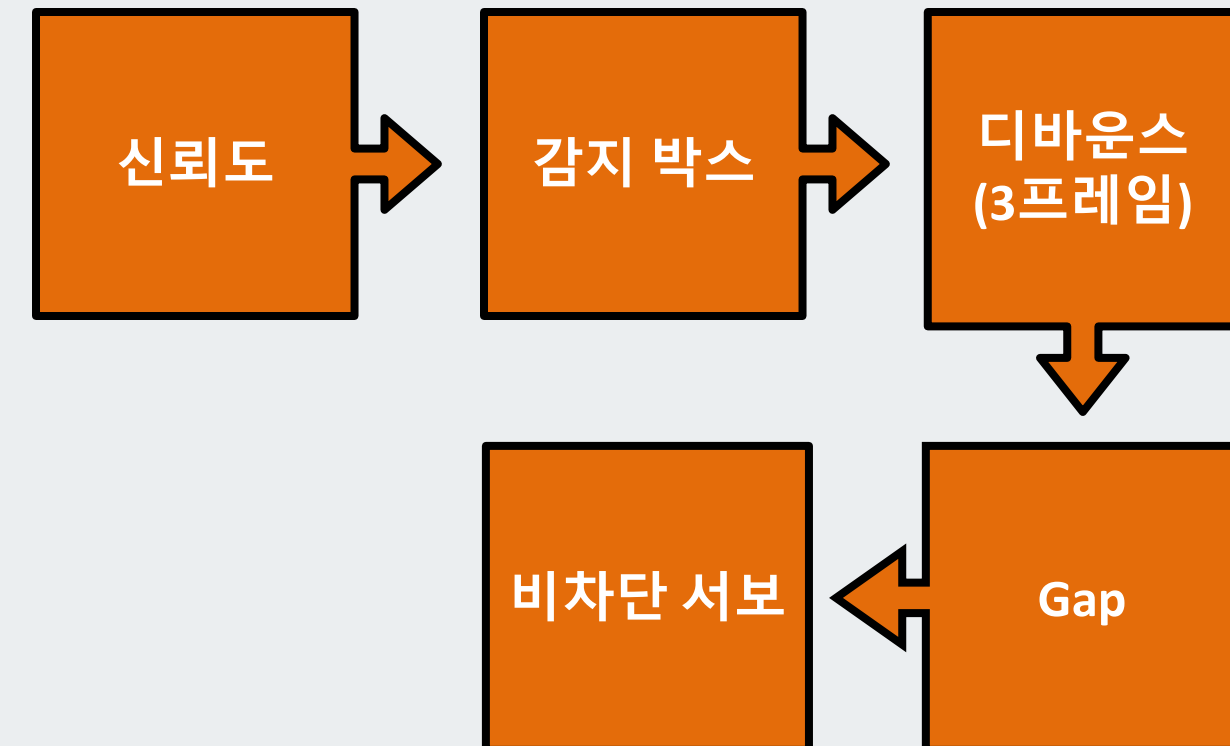
2. 구현 방법 – 제어부

서보

- 동작: 위치제어 SG90 — 신선 90°(통과) / 썩음 50°(좌) / 덜익음 130°(우), 1.3초 유지 후 90° 복귀



- 알고리즘: "딸기 1개 = 정확히 1회 동작"

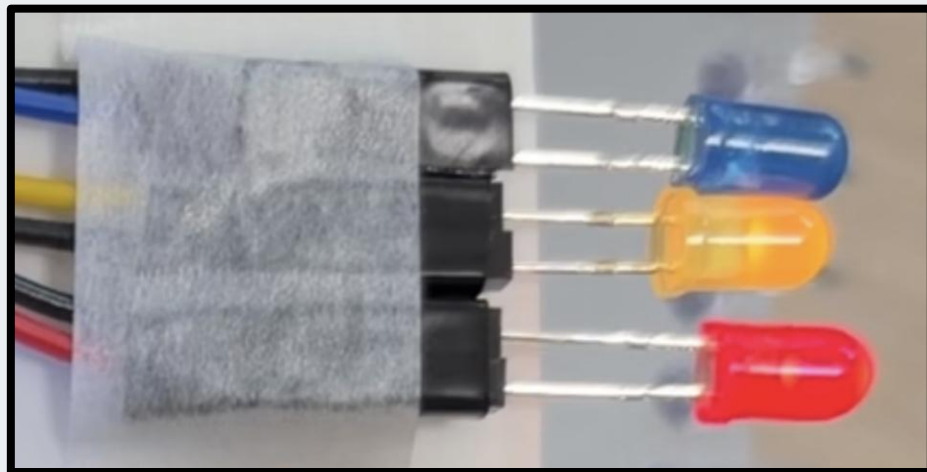


- HW: VCP가 구동하는 부품 = 회전판 서보(SG90) · LED 3색 · LCD
- 전달: D3-G 판단 → IPC → R5 → CAN(0x111, 목표 각도) → VCP가 그 각도로 서보 구동

2. 구현 방법 - 제어부

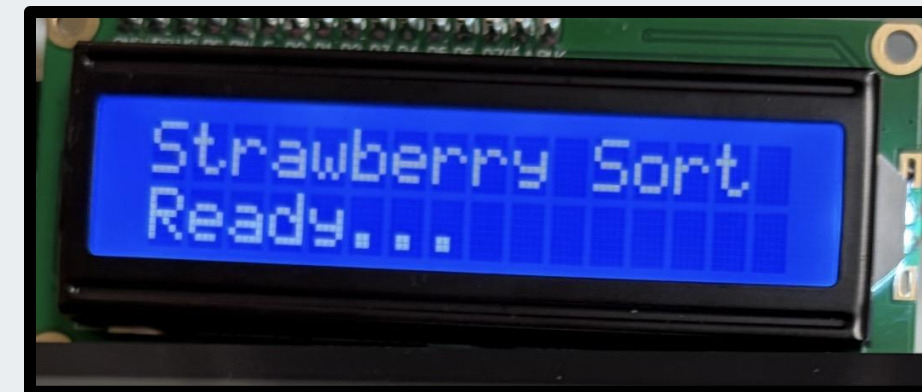
LED — 분류 결과 즉시 표시

- **3색:** 빨강=썩음 / 노랑=덜익음 / 초록=신선 — 감지 즉시 점등 → 1초 후 자동 소등
- **멀티 점등:** 색마다 독립 타이머 → 여러 색 동시 가능, 같은 색 재감지는 타이머만 갱신(중복 전송 x)
- **논블로킹:** 백그라운드로 보내 모터(서보) 타이밍을 절대 안 막음 (모터 우선)



LCD — 누적 통계

- 전체/신선/썩음/덜익음 누적 → 분류 종료(30초 무감지)나 정지 시 표시



2. 구현 방법 - 시스템 통합

```
root@TOPST:/home/topst/Fabless_Education_blink/Step_0# sudo python3 /home/topst/fruit_controller.py
=====
딸기 분류 - D3-G 제어부 (회전판 1서보 / 위치 트리거)
각도 : 신선 90° / 썩음 (원) 50° / 덜익음 (오) 130°, 유지 1.3s
트리거 박스 : X 230~375 AND Y 390~490 / 확인 3프레임 / 서보 유지 1.3s
└ 덜익음 전용 (축소) : X 230~335 AND Y 390~455 (오른쪽·아래 줄여 늦게 트리거)
└ 신선 비토 ON: 박스에서 신선 1회↑ 감지 시 2.0s 서보 차단 (신선 보호)
※ 컨베이어는 외부 제어 - 이 코드는 관여 안 함
LED=감지 즉시 점등 (여러색 가능) · 종료시 OFF / LCD=종료시 결과 / 종료판정=30s 무감지
★오케스트레이션 ON: AI-G(192.168.10.100) SSH로 자동 시작/종료
=====
[NET] D3-G eth0 = 192.168.10.101 고정

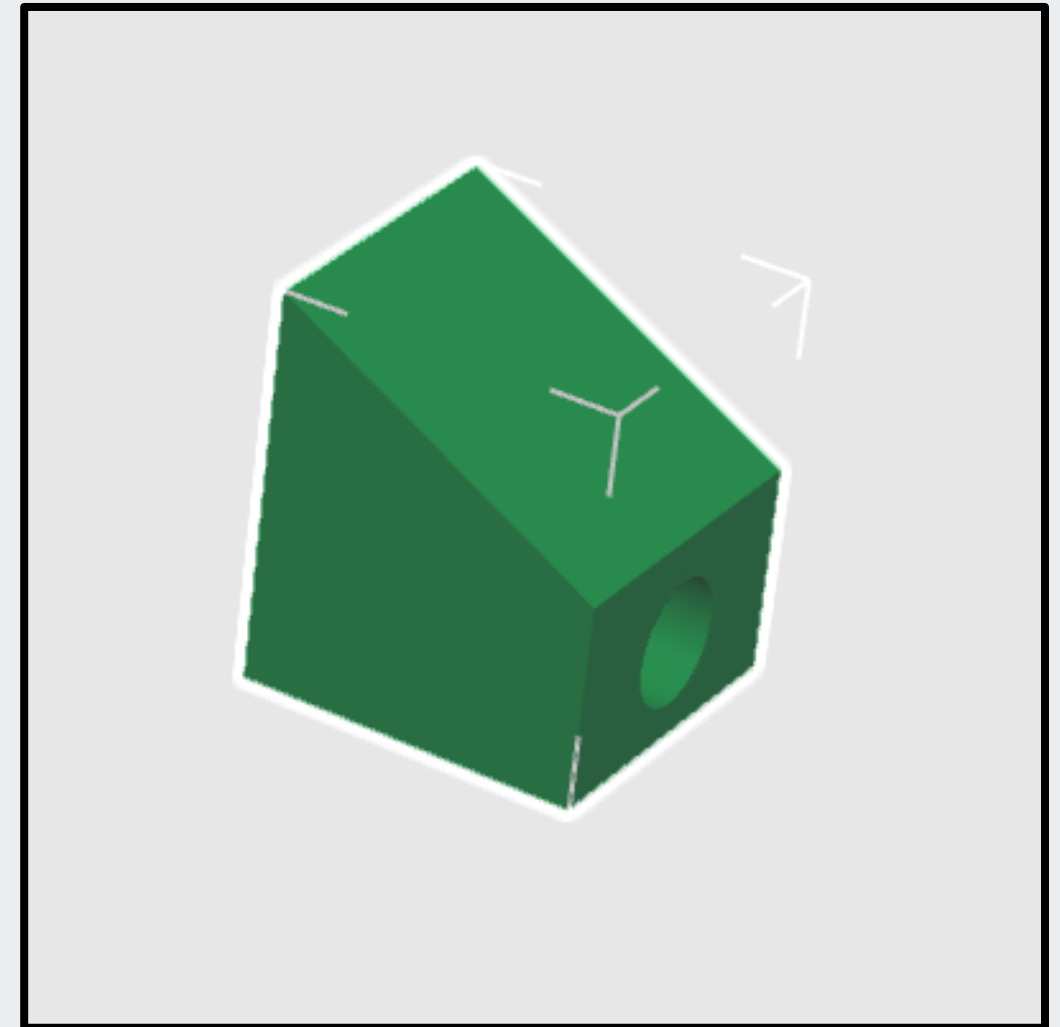
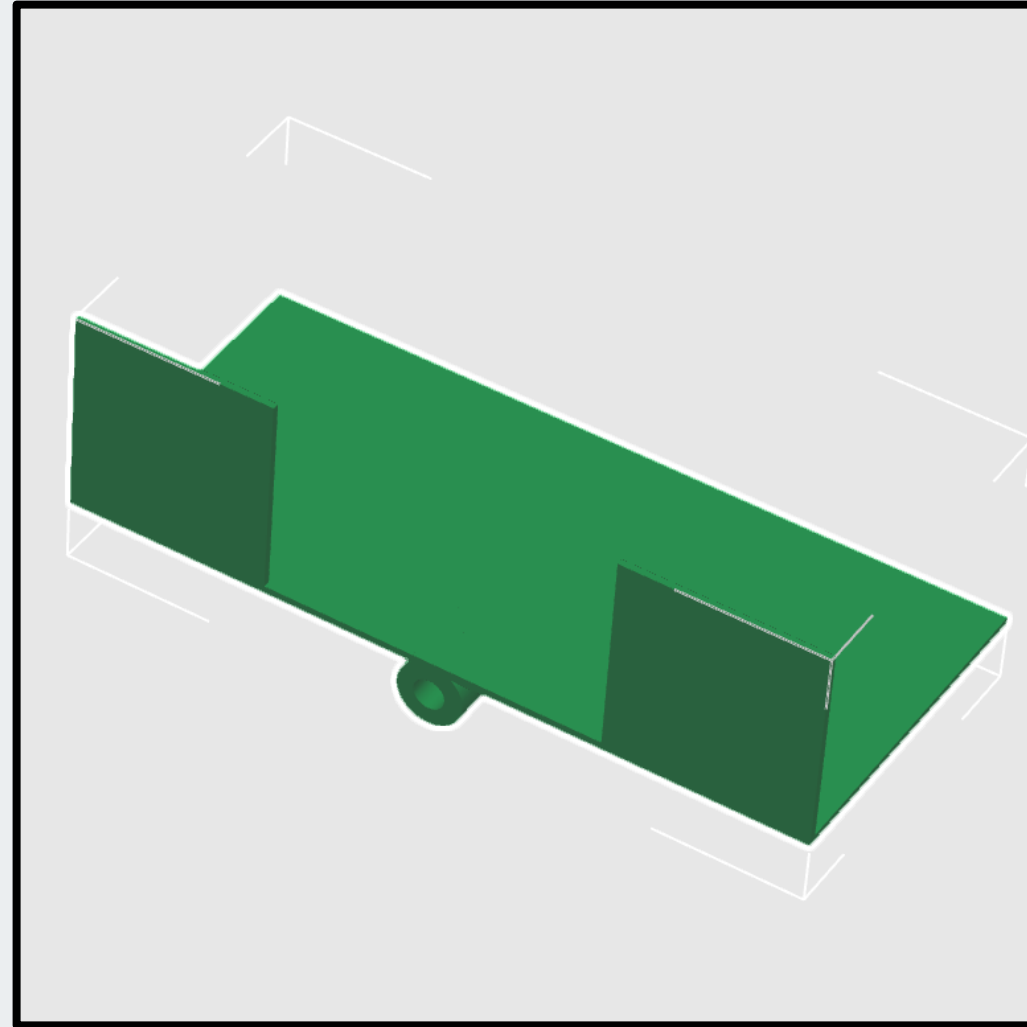
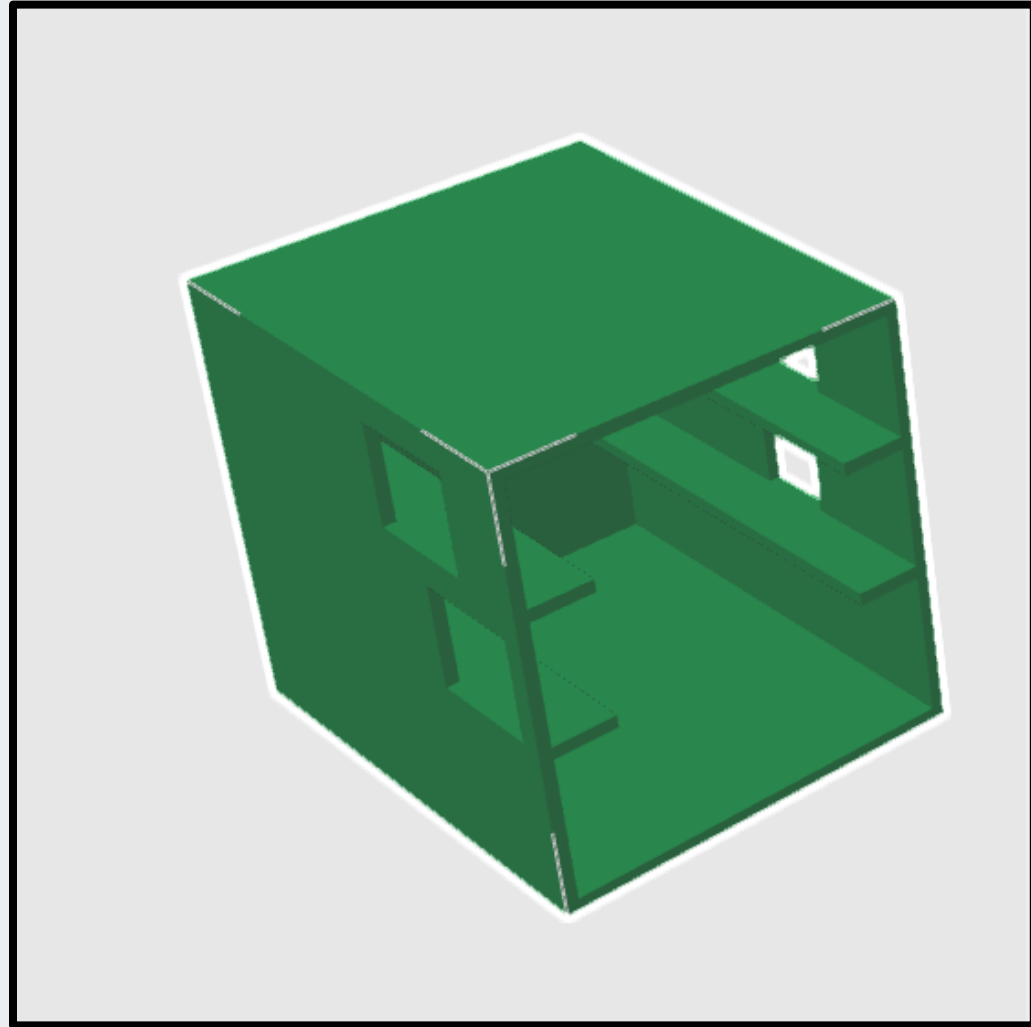
[TCP] AI-G 연결 대기 중... (포트 5000)
[AI-G] 인식 시작 SSH → root@192.168.10.100
=====
딸기 인식 결과 전송 시작
D3-G: 192.168.10.101:5000
=====
[TCP] AI-G 연결됨 : ('192.168.10.100', 52228)
[좌표] 썩음 conf= 69% X=511 Y=583
[좌표] 썩음 conf= 59% X=496 Y=575
[좌표] 썩음 conf= 59% X=486 Y=569
[좌표] 썩음 conf= 69% X=481 Y=570
```

```
[감지] 덜익음 (신뢰도 77%, X=256 Y=452)
[통계] 전체 :3 | 신선 :1(33%) | 썩음 :1(33%) | 덜익음 :1(33%)
[감지] 썩음 (신뢰도 83%, X=364 Y=490)
[통계] 전체 :5 | 신선 :1(20%) | 썩음 :2(40%) | 덜익음 :2(40%)
[감지] 신선 (신뢰도 69%, X=367 Y=467)
→ 신선 : 수평 (90°) 유지 → 통과
[통계] 전체 :6 | 신선 :2(33%) | 썩음 :2(33%) | 덜익음 :2(33%)
```

```
===== 분류 종료 (30s 무감지) → 결과 출력 =====
최종 : 전체 :6 | 신선 :2(33%) | 썩음 :2(33%) | 덜익음 :2(33%)
[AI-G] 인식 중지 SSH (pkill tcnnapp)
[TCP] AI-G 연결 끊김
[INFO] 종료됨
[종료] 30초 무감지 → 시스템 종료. (다음 배치는 다시 실행)
```

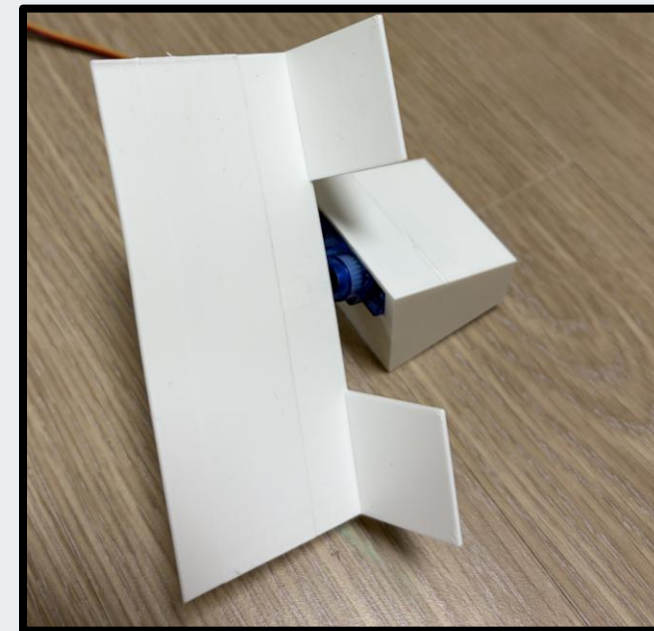
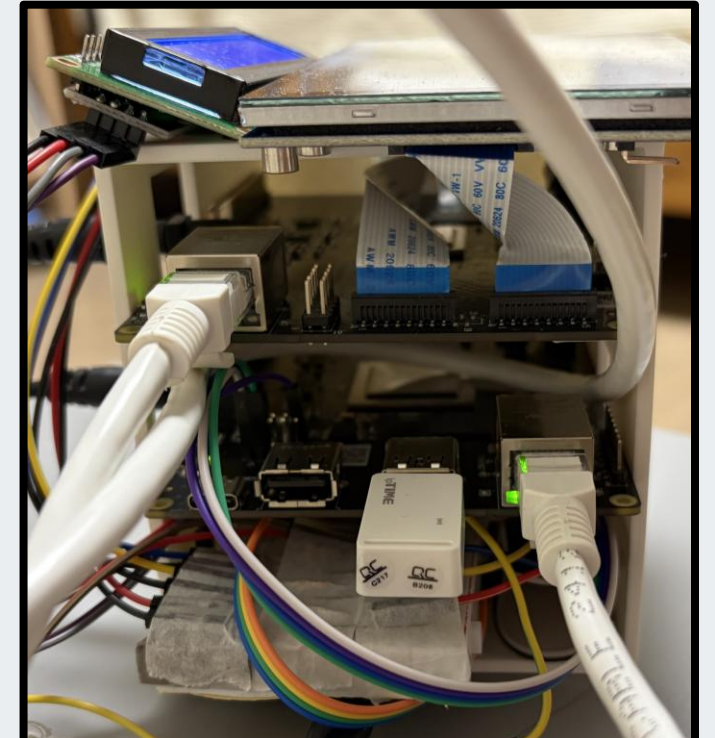
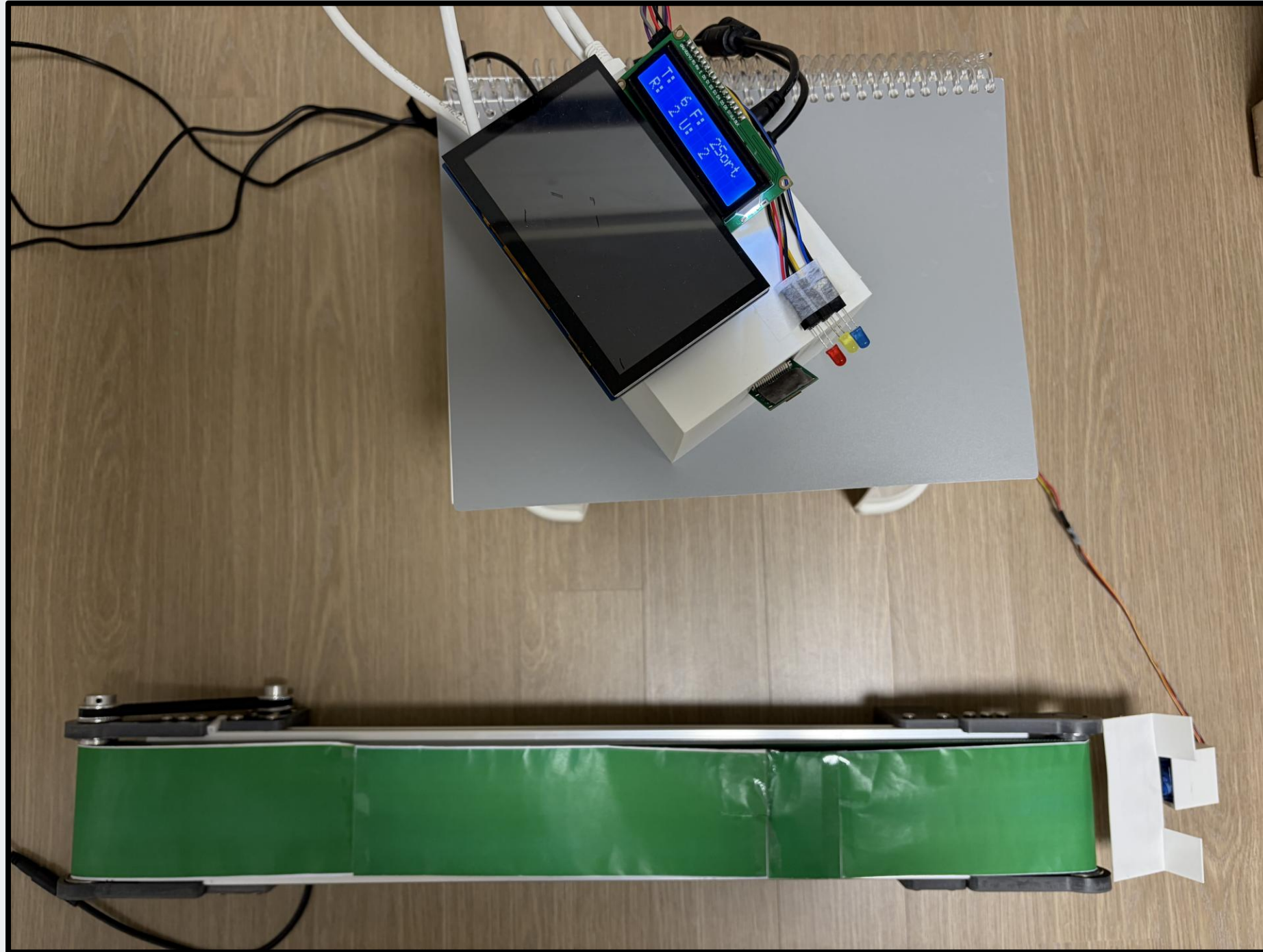
- 전 구간 실동작: 카메라 인식 → 판단 → 회전판·LED·LCD까지 세 보드가 끊김 없이 하나로 (구성을 실제로 통합)
- 무선 한 줄 운영: PC에서 D3-G에 SSH 명령 한 줄 → AI-G까지 자동 기동, 끝나면 자동 종료
- 현장 튜닝(코드화): 박스·각도·시간을 코드 상수로 → 모델 거동 보며 재플래싱 없이 즉시 조정

3. 결과물 - 3D 모델링



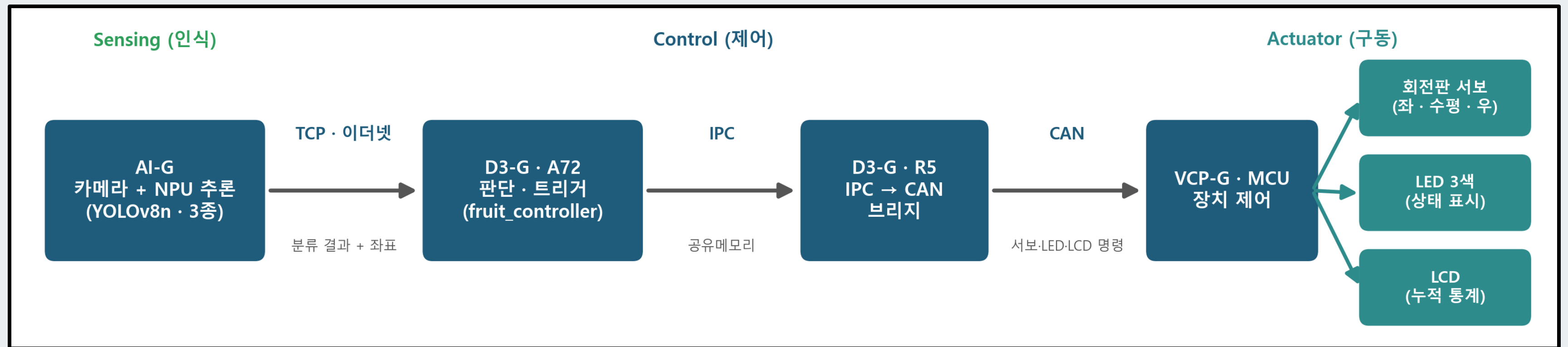
- **컨트롤 타워:** 보드 3개(AI-G·D3-G·VCP)를 3층으로 적층 수납 — 깔끔한 통합 하우징
- **회전판:** SG90에 직결 — 좌·우·수평으로 기울어 종류별 배출(분류 게이트)
- **지지대:** 회전판을 지면과 30° 경사로 받쳐 → 신선 딸기가 굴러 통과

3. 결과물 - 전체 시스템



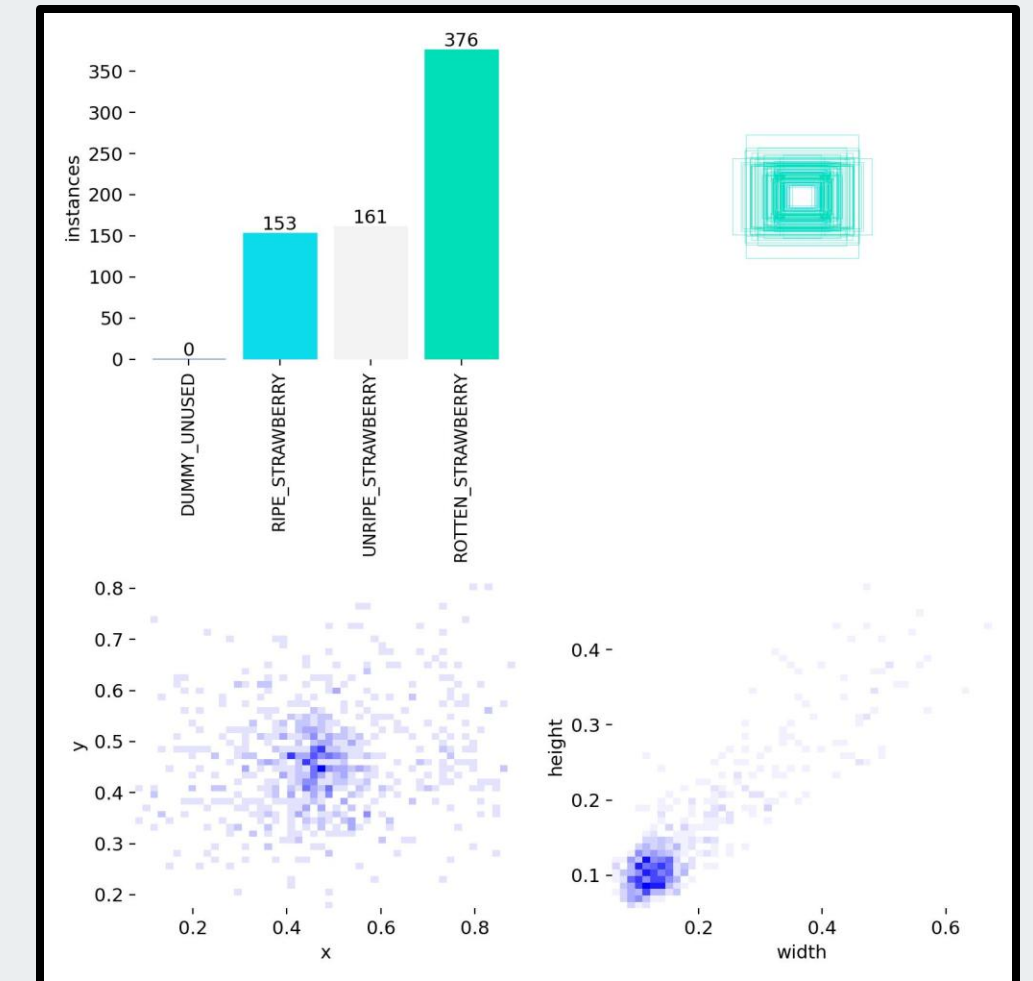
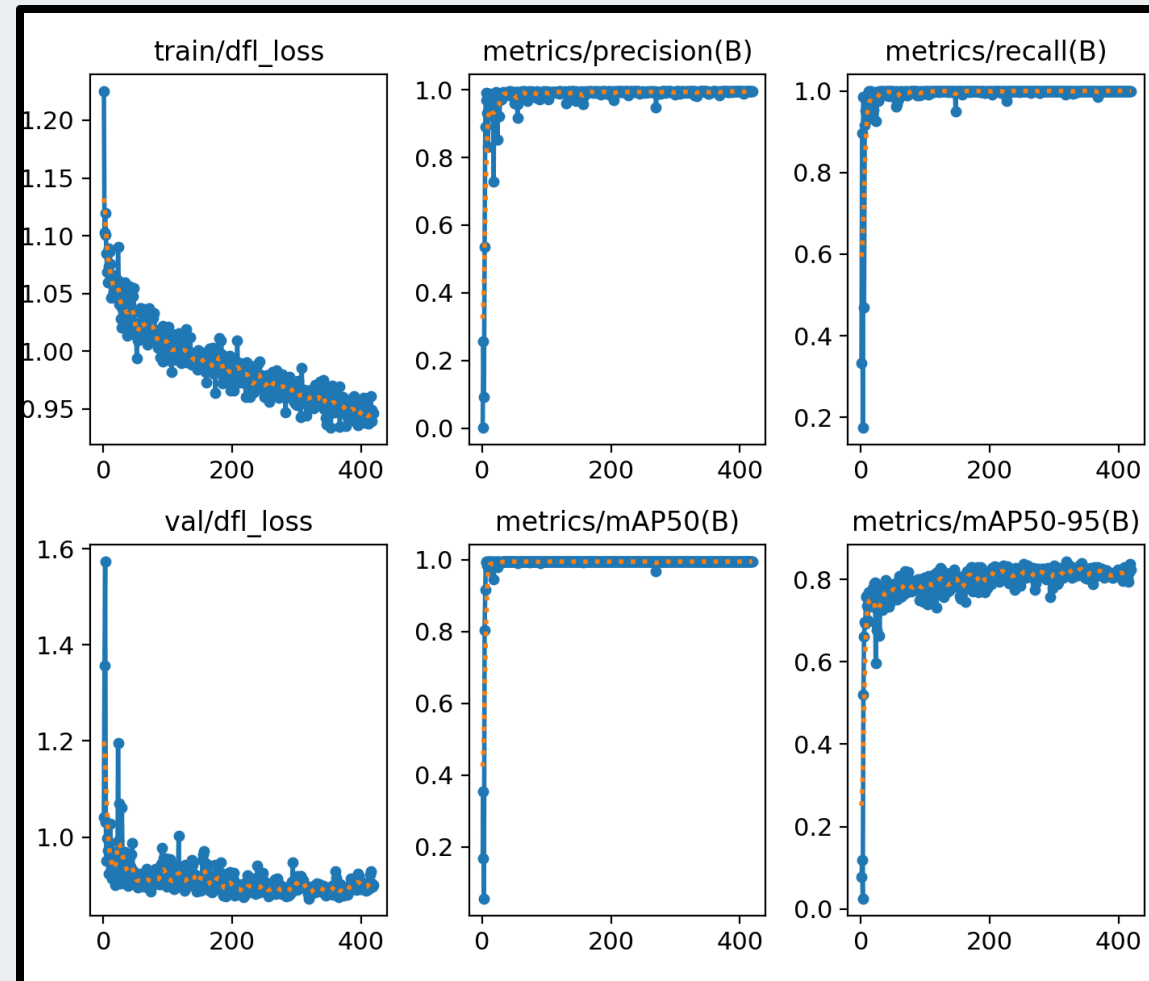
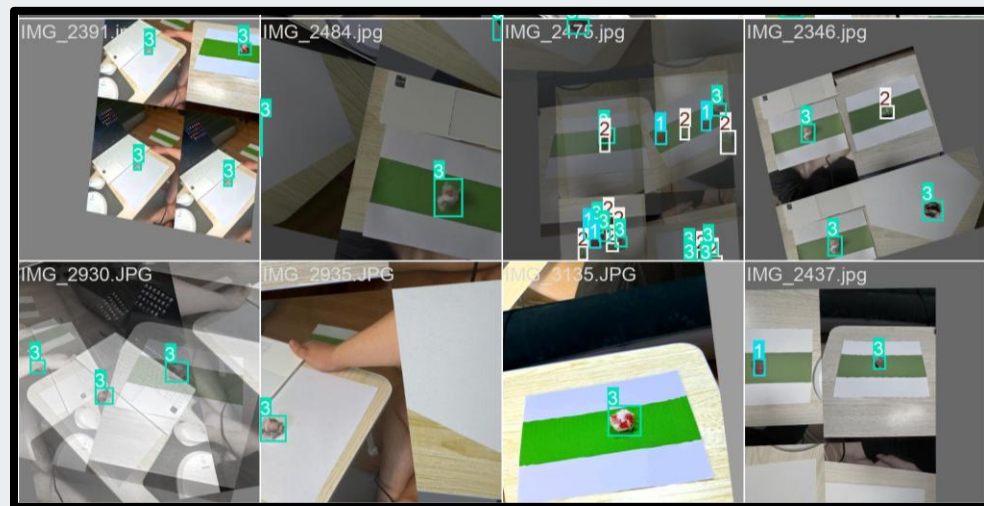
- 물리 구성: 카메라(AI-G) → 컨베이어(외부 구동) → 끝단 회전판(서보)+30° 경사 지지대 + 상태 LED 3색·LCD 통계

3. 결과물 - 전체 시스템



- **작동 흐름:** 전원 ON → 딸기 흘림 → 감지 구간 도달 시 회전판이 좌·우·수평으로 분류 → LED·LCD로 결과 표시

4. 집중한 부분 - AI 학습



- 학습 데이터 직접 제작 (539장): 딸기 신선도용 공개 데이터셋이 없어 → 딸기 모형을 다양한 거리·각도로 직접 촬영·라벨링
- 정확도 끌어올림 (mAP50 0.99): 반복 학습(약 6회)·튜닝으로 분류 정확도 확보
- 증강: mosaic·mixup 증강으로 실환경(거리·각도·움직임) 대응
- NPU class 0 버그 우회: PC에선 정상인데 보드에선 class 0 누락 → 라벨 +1 shift로 해결

4. 집중 한 부분 - !!문제발생!!

- ① 문제: 처음엔 Y좌표만으로 판단해서 딸기가 너무 빨리·예민하게 감지됨
- ② 문제: 덜익음이 학습량이 많아 다른 클래스보다 너무 잘 검출됨
- ③ 문제: 신선 딸기가 가끔 썩음/덜익음으로 오검출됨

→ "감지하면 바로 동작"으로는 신뢰성 부족, 실제 환경에서 마주한 문제

→ 제어 로직으로 보정

4. 집중한 부분 - AI 결과 강화(해결)

```
105 TRIGGER_Y_LO = 390      # 박스 위 경계 (실측: 회전판 자리 딸기 Y≈390~490)
106 TRIGGER_Y_HI = 490      # 박스 아래 경계
107 TRIGGER_X_LO = 230      # 박스 왼쪽 X (실측: 진짜 딸기 X≈230~375, 구석유령 X=118은 제외됨)
108 TRIGGER_X_HI = 375      # 박스 오른쪽 X (오른쪽서 진입 → 넓힐수록 더 일찍 트리거=빠름)

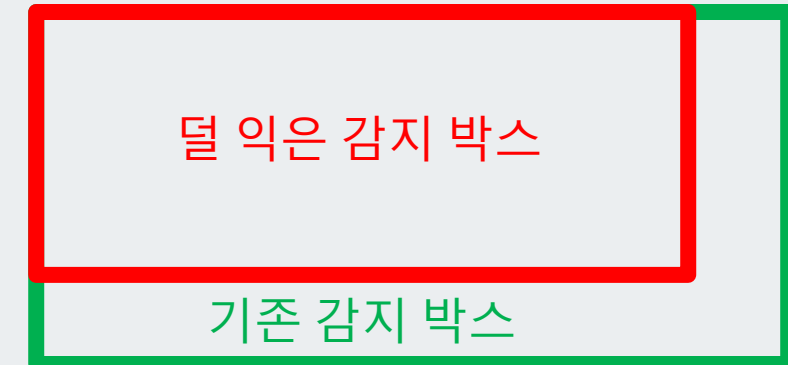
356 in_box = (TRIGGER_X_LO <= cx <= x_hi) and (TRIGGER_Y_LO <= cy <= y_hi)
```

```
113 TRIGGER_X_HI_UNRIPE = 335 # 덜익은 전용 오른쪽 경계 (일반 375 → 40px 안쪽 = 더 늦게 트리거)
114 TRIGGER_Y_HI_UNRIPE = 455 # 덜익은 전용 아래 경계 (일반 490 → 35px 안쪽)

352 if class_id == CLASS_UNRIPE:
353     x_hi, y_hi = TRIGGER_X_HI_UNRIPE, TRIGGER_Y_HI_UNRIPE # 오른쪽·아래만 안쪽으로
354 else:
355     x_hi, y_hi = TRIGGER_X_HI, TRIGGER_Y_HI
356 in_box = (TRIGGER_X_LO <= cx <= x_hi) and (TRIGGER_Y_LO <= cy <= y_hi)
```

```
123 FRESH_VETO      = True      # False면 곰(원래대로 클래스별 독립 트리거)
124 FRESH_VETO_TIME = 2.0      # 신선 1회 감지 후 서보 차단 유지(초) ≈ 딸기 1개 박스 통과시간. 길수록 신선 보호↑

371 # — 신선 비토 설정: 박스에서 신선이 잡히면 잠깐 서보 차단(신선으로 간주) —
372 if FRESH_VETO and class_id == CLASS_FRESH and in_box:
373     if now >= _fresh_veto_until:
374         print(f" [신선 비토] 신선 감지 → {FRESH_VETO_TIME:.1f}s 서보 차단(신선 통과)")
375         _fresh_veto_until = now + FRESH_VETO_TIME
376
377 # — 신선 비토 중이면 씹음/덜익음 발동 안 함(이 딸기는 신선으로 간주 → 그냥 통과) —
378 if (FRESH_VETO and class_id in (CLASS_ROTTER, CLASS_UNRIPE)
379     and now < _fresh_veto_until):
380     if in_box and _armed.get(class_id, True) and _inwin[class_id] >= TRIGGER_CONFIRM:
381         _armed[class_id] = False
382         print(f" [신선 비토] {name} 트리거 무시 → 신선으로 간주 (서보·카운트 X)")
383     return # 서보/카운트/(잘못된)LED 모두 건너뛸
```



- ① (너무 빨리·예민하게 감지) → x·y 좌표를 모두 쓰는 "감지 박스" → 회전판 자리에 왔을 때만 트리거
- ② (덜익음 과검출) → 덜익음만 감지 박스를 작게 → 트리거를 늦춰 다른 클래스와 타이밍 맞춤
- ③ (신선 오분류) → 신선이 한 번이라도 잡히면 신선으로 분류(신선 우선) — 반대 오검출은 없으므로

4. 집중한 부분 - 제어&통합

작동 로직

감지 박스: 회전판 앞 구역(X·Y)에 온 딸기만 — 먼 딸기·유령 제외

디바운스: 연속 3프레임 잡혀야 진짜 — 반짝 튀는 신호 무시

gap 재무장: 사라졌다 다시 와야 새 딸기 — 같은 딸기 중복 방지

비차단 서보: 도는 중 트리거는 건너뛴 — 밀려 헛도는 것 방지



```
[좌 표]  덜 익 음  conf= 69%  X=259  Y=447
[좌 표]  덜 익 음  conf= 59%  X=257  Y=453
[좌 표]  덜 익 음  conf= 77%  X=256  Y=452
→ [덜 익 음 →오 른 쪽] 90°→130° (유 지 1.3s) →90°
```

```
[감 지]  덜 익 음  (신 회 도 77%, X=256 Y=452)
[통 계]  전 체 :3 | 신 선 :1(33%) | 썩 음 :1(33%) | 덜 익 음 :1(33%)
```

통합/운영

튜닝의 코드화: 박스·각도·시간을 코드 상수로 → 모델 동작
보며 재플래싱 없이 즉시 조정

```
96  SERVO_DEFAULT_DEG = 90
97  SERVO_LEFT_DEG     = 50
98  SERVO_RIGHT_DEG    = 130
99  SERVO_HOLD_TIME    = 1.3
```

```
105  TRIGGER_Y_LO = 390
106  TRIGGER_Y_HI = 490
107  TRIGGER_X_LO = 230
108  TRIGGER_X_HI = 375
```


5. 발전 가능성

현재 구현

딸기가 하나씩 간격을 두고 이동

컨베이어 위 딸기 이동이 느림

모형 딸기 데이터 539장 학습

딸기 클래스 3종 외 다른 클래스 존재x
(신선/덜익은/썩은)

처리량

속도

정확도

확장/실용화

발전 요소

여러 개 동시·밀집 감지에도 정확 분류
(AI 다중 객체 인식 향상+ 분류 제어 시스템 정교화)

빠르게 들어오는 객체도 정확 인식
(고속·모션블러 대응 AI 학습+ 빠른 트리거 타이밍)

학습 데이터 추가·클래스 밸런싱
→ 오분류·보정 의존도 감소 + 실물 딸기 대량
테스트로 정량 검증

분류 종류·신선도 등급 세분화 / 완전 무선화 /
시작·종료 버튼 / 통계 대시보드

6. 사용 도구 및 참고자료

- **교육과정:** 텔레칩스 팹리스 실무교육과정 (D01-D10) — TOPST 보드(AI-G·D3-G·VCP) 환경, NPU 워크플로, GPIO·CAN·I2C·PDM, AI-G YOLO 적용
- **AI 학습:** YOLOv8 (Ultralytics) · PyTorch · Google Colab(학습) · LabelImg(라벨링)
- **NPU 배포:** tc-nn-toolkit / Enlight SDK (ONNX → INT8 양자화 → 컴파일) · tcnnapp(NPU 추론)
- **임베디드:** Yocto Linux · FreeRTOS · WSL · FWDN(플래싱)
- **3D 모델링·출력:** Python 파라메트릭 모델링(STEP/STL) · Bambu Studio(X1-Carbon)

5. 고민했던 부분(그냥 빨까 생각중 별로 안중요해서)

- [AI 배포] 보드에서만 class 0 누락 (PC는 정상) → 체계적 진단(PC sim·MD5·raw log) → NPU가 0번을 배경으로 무시 확인 → 라벨 +1 shift로 우회
- [제어] 회전판 1개 + 노이즈로 모터가 헛돌고 밀림 → 비차단 서보(동작 중 스킵) + 박스·디바운스로 "1개=1회"
- [제어] LCD가 백라이트만 켜지고 글자 안 뜸 → I2C 불가능한 핀에 배선된 것을 진단 → 정식 I2C 핀으로 이설
- [통합] "제어가 다 무시된다"를 'log on 필수'로 알았으나 → 콜드부팅 재검증으로 불필요 확정(매 전원 수동단계 제거)